Module Interface Specification for Flexin'

Team 3, Flexin'
Arjun Karthik
Shao (Daniel) Hu
Noah Goldschmied
Peter DeForest
Daniel Franze-Da Silva

November 14, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

See SRS Documentation at [give url —SS]

[Also add any additional symbols, abbreviations or acronyms —SS]

Contents

1 Revision History i

2 Symbols, Abbreviations and Acronyms i

3 Introduction 1

4 Notation 2

5 Module Decomposition 2

6 MIS of Fitness Integration Module 3

6.1 Module . . . 3
6.2 Uses . . . 3
6.3 Syntax . . . 4
6.3.1 Exported Constants . . . 4
6.3.2 Exported Types . . . 4
6.3.3 Exported Access Programs . . . 4
6.4 Semantics . . . 5
6.4.1 State Variables . . . 5
6.4.2 Environment Variables . . . 5
6.4.3 Assumptions . . . 5
6.4.4 Access Routine Semantics . . . 6
6.4.5 Local Functions . . . 7

7 MIS of Persistence Module 7

7.1 Module . . . 7
7.2 Uses . . . 8
7.3 Syntax . . . 8
7.3.1 Exported Constants . . . 8

7.3.2 Exported Access Programs . . . 8
7.4 Semantics . . . 8
7.4.1 State Variables . . . 8
7.4.2 Environment Variables . . . 8
7.4.3 Assumptions . . . 8
7.4.4 Access Routine Semantics . . . 8
7.4.5 Local Functions . . . 8

8 MIS of User and Onboarding Module **8**
8.1 Module . . . 9
8.2 Uses . . . 9
8.3 Syntax . . . 9
8.3.1 Exported Constants . . . 9
8.3.2 Exported Access Programs . . . 9
8.4 Semantics . . . 9
8.4.1 State Variables . . . 9
8.4.2 Environment Variables . . . 10
8.4.3 Assumptions . . . 10
8.4.4 Access Routine Semantics . . . 10
8.4.5 Local Functions . . . 11

9 MIS of Activity Module **11**
9.1 Module . . . 11
9.2 Uses . . . 11
9.3 Syntax . . . 11
9.3.1 Exported Constants . . . 11
9.3.2 Exported Access Programs . . . 12
9.4 Semantics . . . 12
9.4.1 State Variables . . . 12
9.4.2 Environment Variables . . . 12
9.4.3 Assumptions . . . 12
9.4.4 Access Routine Semantics . . . 12
9.4.5 Local Functions . . . 12

10 MIS of Goals Module **12**
10.1 Module . . . 13
10.2 Uses . . . 13
10.3 Syntax . . . 13
10.3.1 Exported Constants . . . 13
10.3.2 Exported Access Programs . . . 14
10.4 Semantics . . . 14
10.4.1 State Variables . . . 14
10.4.2 Environment Variables . . . 14

10.4.3 Assumptions . . . 14
10.4.4 Access Routine Semantics . . . 15
10.4.5 Local Functions . . . 16

11 MIS of Currency Ledger Module **16**
11.1 Module . . . 16
11.2 Uses . . . 16
11.3 Syntax . . . 16
11.3.1 Exported Constants . . . 16
11.3.2 Exported Access Programs . . . 17
11.4 Semantics . . . 17
11.4.1 State Variables . . . 17
11.4.2 Environment Variables . . . 17
11.4.3 Assumptions . . . 17
11.4.4 Access Routine Semantics . . . 17
11.4.5 Local Functions . . . 18

12 MIS of Asset Module **18**
12.1 Module . . . 18
12.2 Uses . . . 18
12.3 Syntax . . . 19
12.3.1 Exported Constants . . . 19
12.3.2 Exported Types . . . 19
12.3.3 Exported Access Programs . . . 20
12.4 Semantics . . . 20
12.4.1 State Variables . . . 20
12.4.2 Environment Variables . . . 20
12.4.3 Assumptions . . . 21
12.4.4 Access Routine Semantics . . . 21
12.4.5 Local Functions . . . 22

13 MIS of Transaction Order Module **23**
13.1 Module . . . 23
13.2 Uses . . . 23
13.3 Syntax . . . 24
13.3.1 Exported Constants . . . 24
13.3.2 Exported Types . . . 24
13.3.3 Exported Access Programs . . . 25
13.4 Semantics . . . 25
13.4.1 State Variables . . . 25
13.4.2 Environment Variables . . . 25
13.4.3 Assumptions . . . 26
13.4.4 Access Routine Semantics . . . 26

13.4.5 Local Functions . . . 28

14 MIS of Portfolio Module **28**
14.1 Module . . . 28
14.2 Uses . . . 28
14.3 Syntax . . . 28
14.3.1 Exported Constants . . . 28
14.3.2 Exported Access Programs . . . 29
14.4 Semantics . . . 29
14.4.1 State Variables . . . 29
14.4.2 Environment Variables . . . 29
14.4.3 Assumptions . . . 29
14.4.4 Access Routine Semantics . . . 29
14.4.5 Local Functions . . . 29

15 MIS of Leaderboard and Friends Module **29**
15.1 Module . . . 29
15.2 Uses . . . 29
15.3 Syntax . . . 30
15.3.1 Exported Constants . . . 30
15.3.2 Exported Access Programs . . . 30
15.4 Semantics . . . 30
15.4.1 State Variables . . . 30
15.4.2 Environment Variables . . . 30
15.4.3 Assumptions . . . 30
15.4.4 Access Routine Semantics . . . 30
15.4.5 Local Functions . . . 30

16 MIS of Progress Reporting Module **30**
16.1 Module . . . 30
16.2 Uses . . . 31
16.3 Syntax . . . 31
16.3.1 Exported Constants . . . 31
16.3.2 Exported Access Programs . . . 31
16.4 Semantics . . . 31
16.4.1 State Variables . . . 31
16.4.2 Environment Variables . . . 31
16.4.3 Assumptions . . . 31
16.4.4 Access Routine Semantics . . . 31
16.4.5 Local Functions . . . 31

17 MIS of UI Screens Module **31**
17.1 Module . . . 32
17.2 Uses . . . 32
17.3 Syntax . . . 32
17.3.1 Exported Constants . . . 32
17.3.2 Exported Access Programs . . . 32
17.4 Semantics . . . 32
17.4.1 State Variables . . . 32
17.4.2 Environment Variables . . . 32
17.4.3 Assumptions . . . 32
17.4.4 Access Routine Semantics . . . 32
17.4.5 Local Functions . . . 32

18 MIS of Notification Module **33**
18.1 Module . . . 33
18.2 Uses . . . 33
18.3 Syntax . . . 33
18.3.1 Exported Constants . . . 33
18.3.2 Exported Access Programs . . . 33
18.4 Semantics . . . 33
18.4.1 State Variables . . . 33
18.4.2 Environment Variables . . . 33
18.4.3 Assumptions . . . 33
18.4.4 Access Routine Semantics . . . 33
18.4.5 Local Functions . . . 34

19 MIS of Market Simulation Module **35**
19.1 Module . . . 35
19.2 Uses . . . 35
19.3 Syntax . . . 35
19.3.1 Exported Constants . . . 35
19.3.2 Exported Types . . . 36
19.3.3 Exported Access Programs . . . 36
19.4 Semantics . . . 37
19.4.1 State Variables . . . 37
19.4.2 Environment Variables . . . 37
19.4.3 Assumptions . . . 38
19.4.4 Access Routine Semantics . . . 38
19.4.5 Local Functions . . . 40

20 MIS of API Client Module **42**
20.1 Module . . . 42
20.2 Uses . . . 42
20.3 Syntax . . . 42
20.3.1 Exported Constants . . . 42
20.3.2 Exported Types . . . 43
20.3.3 Exported Access Programs . . . 43
20.4 Semantics . . . 44
20.4.1 State Variables . . . 44
20.4.2 Environment Variables . . . 45
20.4.3 Assumptions . . . 45
20.4.4 Access Routine Semantics . . . 45
20.4.5 Local Functions . . . 47

21 MIS of Authentication and Security Module **48**
21.1 Module . . . 48
21.2 Uses . . . 48
21.3 Syntax . . . 48
21.3.1 Exported Constants . . . 48
21.3.2 Exported Types . . . 48
21.3.3 Exported Access Programs . . . 49
21.4 Semantics . . . 50
21.4.1 State Variables . . . 50
21.4.2 Environment Variables . . . 50
21.4.3 Assumptions . . . 51
21.4.4 Access Routine Semantics . . . 51
21.4.5 Local Functions . . . 54

22 Appendix — Generative AI Use Disclosure **55**
22.1 AI Tools Used . . . 55
22.2 How and Where Used . . . 55
22.3 Verification of AI-Generated Content . . . 56

23 Appendix — Reflection **56**

3 Introduction

The following document details the Module Interface Specifications for [Fill in your project name and description —SS]

Complementary documents include the System Requirement Specifications and Module Guide. The full documentation and implementation can be found at .... [provide the url for your repo —SS]

4 Notation

[You should describe your notation. You can use what is below as a starting point. —SS]

The structure of the MIS for modules comes from **?**, with the addition that template modules have been adapted from **?**. The mathematical notation comes from Chapter 3 of **?**. For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1|c_2 \Rightarrow r_2|...|c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by Flexin'.

Data Type	Notation	Description
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in (-∞, ∞)
natural number	$\mathbb{N}$	a number without a fractional component in [1, ∞)
real	$\mathbb{R}$	any number in (-∞, ∞)

The specification of Flexin' uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In addition, Flexin' uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification. The symbol $\perp$ means that there is no value, so a value of type $T \cup \{\perp\}$ is either a T or absent.

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

Level 1	Level 2
Hardware-Hiding Module	Fitness Integration Module
Behaviour-Hiding Module	Persistence Module User and Onboarding Module Activity Module Goals Module Currency Ledger Module Asset Module Transaction Order Module Portfolio Module Leaderboard and Friends Module Progress Reporting Module UI Screens Module Notification Module Market Simulation Module API Client Module
Software Decision Module	Authentication and Security Module

Table 1: Module Hierarchy

6 MIS of Fitness Integration Module

This module is M1 in the Module Guide.

6.1 Module

FitnessIntegration

This module is an abstract data type with one instance for each fitness source: healthConnect and garmin. Every instance exports the same access programs, so the rest of the app reads fitness data without knowing which source it came from. The semantics below are given for the Health Connect instance. The Garmin instance follows the same rules, reading Garmin data through Garmin Connect IQ in place of Health Connect.

6.2 Uses

None. This module runs on the phone and reaches each source only through the source's own SDK or API (SC-04), which is an environment variable of this module.

6.3 Syntax

6.3.1 Exported Constants

None. The module uses the following local constants, which are not exported.

Constant	Value	Meaning
Permissions	read Steps, read ExerciseSession	The only Health Connect permissions requested
MaxCadence	250 steps per minute, sustained	Steps from a record whose average rate is above this are reported as unverified (SEC-INT-01). The value is the SRS constant of the same name.

6.3.2 Exported Types

- FitnessSource = {healthConnect, garmin}
- Time: a point in time, independent of time zone
- LocalDate: a calendar date in the device's time zone
- ActivitySample = tuple of (*source*: FitnessSource, *id*: String, *type*: String, *startTime*: Time, *endTime*: Time). A workout as reported by a source. *id* is the source's own identifier for the workout and does not change between reads (PR-PA-03).
- DailySteps = tuple of (*source*: FitnessSource, *date*: LocalDate, *steps*: $\mathbb{Z}$, *unverifiedSteps*: $\mathbb{Z}$). The steps from one source on one local day. *unverifiedSteps* are the steps from records faster than MaxCadence; they are not part of *steps* and earn no currency unless verified (SEC-INT-01). A later DailySteps for the same source and date replaces an earlier one.

6.3.3 Exported Access Programs

Name	In	Out	Exceptions
isAvailable	-	$\mathbb{B}$	-
requestPermissions	-	$\mathbb{B}$	-
fetchActivities	*since*: Time	seq of ActivitySample	ReadError
fetchDailySteps	*since*: Time	seq of DailySteps	ReadError
activityKey	*a*: ActivitySample	String	-
dailyStepsKey	*d*: DailySteps	String	-

ReadError: the source cannot be read, because it is not available on the device or a permission in Permissions has not been granted.

6.4 Semantics

6.4.1 State Variables

- *source*: FitnessSource, the source this instance reads, fixed when the instance is created

The module stores no fitness data (SEC-PRV-01).

6.4.2 Environment Variables

- *hc*: Health Connect on the device. It provides:
  - *hc.available*: $\mathbb{B}$, whether Health Connect can be used on the device.
  - *hc.granted*: the set of permissions the user has granted to the app.
  - The Steps and ExerciseSession records written by other apps. Each record r has a start and end Time, an identifier $r.id$, the app that wrote it $r.origin$, and how it was recorded $r.method$. A Steps record also has a step count $r.count$, and an ExerciseSession record has an exercise type.
- *clock*: the device clock; $now()$ returns the current Time.
- *tz*: the device time zone, which decides the local day that a Time falls on.
- Garmin Connect IQ, Garmin's published SDK, which the Garmin instance reads in place of *hc* (SC-04).

6.4.3 Assumptions

- $since \leq now()$ for every fetch.
- *tz* does not change during a fetch.
- $r.method$ is reported by the app that wrote the record, so a manual entry is recognised only when that app marks it as one. Invented steps that are not marked as manual are caught only when they are faster than MaxCadence.
- Workouts that overlap in time, including the same workout recorded by two sources, are detected by the Activity Module (Section 9), which holds the workouts from earlier syncs (SEC-INT-01, PR-PA-03). This module reports every workout it reads.
- Health Connect returns records from at most about 30 days before the first permission grant. This covers OfflineSyncWindow (PR-RFT-01).

- The backend uses the keys from activityKey and dailyStepsKey so that each workout is recorded once and each daily total replaces the previous one, with currency awarded only for the increase.

6.4.4 Access Routine Semantics

isAvailable():

- output: $out := hc.available$

requestPermissions():

- transition: if $hc.available$, the user is asked to grant Permissions, and $hc.granted$ is updated with the user's choice.
- output: $out := hasPermissions()$, so a partial grant gives false.

fetchActivities(*since*):

- output: one ActivitySample for each ExerciseSession record r in hc between *since* and $now()$ with $r.id \neq \bot \wedge \neg manual(r)$, giving its identifier, exercise type, and start and end Time. Manual entries are left out (SC-04).
- exception: $\neg hasPermissions() \Rightarrow$ ReadError

fetchDailySteps(*since*):

- output: let $t = now()$ and let R be the Steps records in hc between *since* and t with $counted(r)$. Let $V = \{r \in R \mid \neg tooFast(r)\}$ and $U = \{r \in R \mid tooFast(r)\}$. Then $out := \langle (source, d, dayTotal(V, d), dayTotal(U, d)) \mid d \in days(since, t) \rangle$. There is one DailySteps for each local day from *since* to today, with 0 for a day with no steps. Steps are dated by when they happened, not when they were read (PR-PA-05).
- exception: $\neg hasPermissions() \Rightarrow$ ReadError

activityKey(a):

- output: a String determined by $(a.source, a.id)$ alone, different for different pairs

dailyStepsKey(d):

- output: a String determined by $(d.source, d.date)$ alone, different for different pairs

So a sync that repeats or overlaps an earlier one never reports the same activity as new twice (PR-PA-03).

6.4.5 Local Functions

For a Steps record r, $len(r)$ is its length in minutes, and $overlap(r, d)$ is the time that r shares with local day d, in minutes.

- *hasPermissions*: $\to \mathbb{B}$
  $hasPermissions() \equiv hc.available \land \mathsf{Permissions} \subseteq hc.granted$

- *manual*: record $\to \mathbb{B}$
  $manual(r) \equiv r.method = \text{MANUAL_ENTRY}$

- *counted*: Steps record $\to \mathbb{B}$
  $counted(r) \equiv \neg manual(r) \land len(r) > 0 \land r.count \geq 0$. Steps of other records are not reported at all, not even as unverified (SC-04).

- *tooFast*: Steps record $\to \mathbb{B}$
  $tooFast(r) \equiv r.count/len(r) > \mathsf{MaxCadence}$, the rate sustained over the record (SEC-INT-01).

- *appTotal*: set of Steps record $\times$ String $\times$ LocalDate $\to \mathbb{R}$
  $appTotal(R, o, d) \equiv \sum_{r \in R\,:\, r.origin = o} r.count \cdot overlap(r, d)/len(r)$, the steps that app o recorded on day d, taking each record's steps as spread evenly over its time.

- *dayTotal*: set of Steps record $\times$ LocalDate $\to \mathbb{Z}$
  The steps of R on day d, with steps that more than one app recorded at the same time counted once (PR-PA-03). It satisfies

  $$\max_o appTotal(R, o, d) \leq dayTotal(R, d) \leq \sum_o appTotal(R, o, d)$$

  up to rounding. How overlapping records are combined is a secret of this module (see the Module Guide).

- *localDate*: Time $\to$ LocalDate
  The local calendar date of a Time in tz.

- *days*: Time $\times$ Time $\to$ seq of LocalDate
  $days(s, t) \equiv$ every local date from $localDate(s)$ to $localDate(t)$ inclusive, in order.

7 MIS of Persistence Module

This module is M2 in the Module Guide.

7.1 Module

Persistence

7.2 Uses

[Modules this module uses —SS]

7.3 Syntax

7.3.1 Exported Constants

[Exported constants, or None —SS]

7.3.2 Exported Access Programs

Name	In	Out	Exceptions
[accessProg —SS]	-	-	-

7.4 Semantics

7.4.1 State Variables

[State variables, or None —SS]

7.4.2 Environment Variables

[Environment variables, or None —SS]

7.4.3 Assumptions

[Assumptions, or None —SS]

7.4.4 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

7.4.5 Local Functions

[Local functions, or None —SS]

8 MIS of User and Onboarding Module

This module is M3 in the Module Guide.

8.1 Module

User and Onboarding Module

8.2 Uses

- Persistence module: stores and send user information as needed. Checks to see if a user exists.

8.3 Syntax

8.3.1 Exported Constants

Name	Value	Meaning
MinAge	18	Minimum age to register (years)
MinPassLength	8	Minimum length of a password string

8.3.2 Exported Access Programs

Name	In	Out	Exceptions
register	email: String, password: string, dob: Date	userID: int	InvalidEmail, EmailTaken, Underage
getUserProfile	userID: int	UserProfile	NoAccount
updateUserPassword	userID: int, password: string, newPassword: string	N/A	NoAccount
updateUserEmail	userID: int, email: string, newEmail: string	N/A	NoAccount, InvalidEmail, EmailTaken
submitFinance	userID: int, financeAnswers: list[string]	N/A	NoAccount, InvalidResponse
submitFitness	userID: int, fitnessAnswers: list[string]	N/A	NoAccount, InvalidResponse
isOnboarded	userID: int	$\mathbb{B}$	NoAccount

8.4 Semantics

8.4.1 State Variables

None, user information will be saved to the database (Persistence Module).

8.4.2 Environment Variables

- *clock*: the system clock; *now*() returns the current Time.

8.4.3 Assumptions

The User and Onboarding Module will rely on:

- The Persistence Module is active and can be accessed.
- Valid response has been provided by the Authentication and Security Module.

8.4.4 Access Routine Semantics

The following apply to every access routine below.

- An exception leaves the state unchanged.

register(email, password, dob)

- transition: the persistence module will store the user's valid inputted credentials
- output: the new UserID.
- exception: InvalidEmail $\longrightarrow$ **validEmail** does not validate the email, EmailTaken $\longrightarrow$ an email is already in use, Underage $\longrightarrow$ user dob $\leq$ MinAge

getUserProfile(userID)

- output: the profile object attached to userID
- exception: NoAccount $\longrightarrow$ no userID found that matches the input

updateUserPassword(userID, password, newPassword)

- transition: the persistence module will replace the user's password with the user's new valid inputted password
- exception: NoAccount $\longrightarrow$ no userID found that matches the input

updateUserEmail(userID, email, newEmail)

- transition: the persistence module will replace the user's email with the user's new valid inputted email
- exception: NoAccount $\longrightarrow$ no userID found that matches the input

submitFinance(userID, financeAnswers)

- transition: the persistence module will store the users inputted responses

- exception:NoAccount $\longrightarrow$ no userID found that matches the input

submitFitness(userID, fitnessAnswers)

- transition: the persistence module will store the users inputted responses
- exception: NoAccount $\longrightarrow$ no userID found that matches the input

isOnboarded(userID)

- output: if the user has completed the onboarding surveys for both Fitness and Finance.
- exception: NoAccount $\longrightarrow$ no userID found that matches the input

8.4.5 Local Functions

- validEmail(email: string) $\longrightarrow \mathbb{B}$: ensures email is valid and exists
- validPassword(password: string) $\longrightarrow \mathbb{B}$: ensures password contains any needed characters and length(password) $\geq$ MinPassLength
- validDOB(dob: Date)$\longrightarrow \mathbb{B}$: ensures user's age $\geq$ MinAge

9 MIS of Activity Module

This module is M4 in the Module Guide.

9.1 Module

Activity

9.2 Uses

- Persistence Module: stores user activity, current goals and progress toward those goals.

9.3 Syntax

9.3.1 Exported Constants

Name	Value	Meaning
RefreshRate	120 seconds	The amount of time between a sync call to update the fitness data shown in the app from the outside source tracking the activity

9.3.2 Exported Access Programs

Name	In	Out	Exceptions
getActiviy	userID: int, date: Date	activityStats: Map[activity: string $\longrightarrow$ value: int]	NoAccount, InvalidDate

9.4 Semantics

9.4.1 State Variables

None, the persistence module will store user activity records.

9.4.2 Environment Variables

- *clock*: the system clock; *now*() returns the current Time.

9.4.3 Assumptions

The Currency Ledger Module will rely on:

- The Persistence Module is active and can be accessed.
- Valid response has been provided by the Authentication and Security Module.
-
- Each record carries a stable identifier and the time it occurred on the source device. Records have already been validated by the fitness data provider and are trusted.

9.4.4 Access Routine Semantics

getActiviy(userID):

- output: returns a map of activity and tracked data attached to that activity.
- exception: NoAccount $\longrightarrow$ no userID found that matches the input, InvalidDate $\longrightarrow$ checks to ensure a valid date has been entered

9.4.5 Local Functions

None, this module is a getter and has no write capabilities.

10 MIS of Goals Module

This module is M5 in the Module Guide.

10.1 Module

Goals (M5)

10.2 Uses

- Persistence Module: stores user fitness activity for the user.
- Activity Module: retrieves fitness activity to use in the app.
- Currency Ledger Module: the goals module tracks progress toward goals and will be rewarded by the currency ledger module.

10.3 Syntax

10.3.1 Exported Constants

None.

10.3.2 Exported Access Programs

Name	In	Out	Exceptions
getAllGoals	userID: int	goalMap: Map[activityType: string ⟶ activityMeasurement: int]	NoAccount
createGoal	userID: int, activityType: string, activityMeasurement: int, period: startDate ⟶ endDate	goalID: int	NoAccount, InvalidActivityType, InvalidActivityMeasurement, InvalidPeriod
deleteGoal	userID: int, goalID: int	N/A	NoAccount, NoGoal
updateGoalMeasurement	userID: int, goalID: int, activityMeasurement: int	N/A	NoAccount, NoGoal, InvalidActivityMeasurement, GoalLocked
updateGoalPeriod	userID: int, goalID: int, period: period	N/A	NoAccount, NoGoal, InvalidPeriod, GoalLocked
collectReward	userID: int, goalID: int	N/A	NoAccount, NoGoal, GoalLocked

10.4 Semantics

10.4.1 State Variables

None, information will be stored in the database (Persistence Module)

10.4.2 Environment Variables

- *clock*: the system clock; *now*() returns the current Time.

10.4.3 Assumptions

- The Persistence Module is active and can be accessed.
- Valid response has been provided by the Authentication and Security Module.

- Valid data from the Activity Module is retrieved.

10.4.4 Access Routine Semantics

getAllGoals(userID):

- output: a map of the users activity type and the value attached to the actvity.
- exception: NoAccount $\longrightarrow$ no userID found that matches the input.

createGoal(userID, activityType, activityMeasurement):

- transition: a new goal will be added to the users goal data.
- output: the new goalID.
- exception: NoAccount $\longrightarrow$ no userID found that matches the input, InvalidActivityType $\longrightarrow$ a non-supported activity type, InvalidActivityMeasurement $\longrightarrow$ the measurement value inputted is activityMeasurement < 0.

deleteGoal(userID, goalID):

- transition: the goal will be removed from the users goal data.
- exception: NoAccount $\longrightarrow$ no userID found that matches the input, NoGoal $\longrightarrow$ no goalID found that matches the input.

updateGoalMeasurement(userID, goalID, activityMeasurement):

- transition: updates the measurement on an already started goal.
- exception: NoAccount $\longrightarrow$ no userID found that matches the input, NoGoal $\longrightarrow$ no goalID found that matches the input, InvalidActivityMeasurement $\longrightarrow$ the measurement value inputted is activityMeasurement < 0, GoalLocked $\longrightarrow$ if the goalID is locked the goal is no longer able to be changed.

updateGoalPeriod(userID, goalID, period):

- transition: updates the period of an already started goal.
- exception: NoGoal $\longrightarrow$ no goalID found that matches the input, InvalidPeriod $\longrightarrow$ the period ends before it starts, GoalLocked $\longrightarrow$ if the goalID is locked the goal is no longer able to be changed.

.
collectReward(userID, goalID):

- transition: notifies to the Currency Ledger Module to add the reward to the user's account. After a goal is completed and the reward collected, the goal will no longer be accessible for anything but viewing.

- exception: NoAccount $\longrightarrow$ no userID found that matches the input, NoGoal $\longrightarrow$ no goalID found that matches the input, GoalLocked $\longrightarrow$ if the goalID is locked the goal is no longer able to be changed.

.

10.4.5 Local Functions

Name	Type	Meaning
checkGoalCompletion	$\mathbb{B}\mathbb{B}$	If the user activity $\geq$ current goal activity measurement values, then the goal is complete.
isLocked	userID: int $\times$ goalID: int $\rightarrow \mathbb{B}$	True iff the goal's reward has been collected (its status is rewarded). A locked goal can only be viewed.

11 MIS of Currency Ledger Module

This module is M6 in the Module Guide.

11.1 Module

Currency Ledger Module

11.2 Uses

- Transaction Order Module: uses the Currency Ledger to add currency to, or remove currency from, a user's account based on the action that has taken place.
- Leaderboard and Friends Module: uses the Currency Ledger to allow users to compete against one another.
- Persistence Module: stores user balances and records the movement of currency into and out of a user's account.

11.3 Syntax

11.3.1 Exported Constants

None, information will be store in the database (Persistence Module) and the Currency Ledger Module will rely on the database for information retrieval

11.3.2 Exported Access Programs

Name	In	Out	Exceptions
getBalance	UserID: int	acctBalance: BigDecimal	NoAccount
addBalance	UserID: int, amount: BigDecimal	acctBalance: BigDecimal	NoAccount, InvalidAmount
subtractBalance	UserID: int, amount: BigDecimal	acctBalance: BigDecimal	NoAccount, InvalidAmount

11.4 Semantics

11.4.1 State Variables

None, user balances will be saved to the database (Persistence Module).

11.4.2 Environment Variables

- *clock*: the system clock; *now*() returns the current Time.

11.4.3 Assumptions

The Currency Module will rely on:

- The Persistence Module is active and can be accessed.
- The Transaction Order Module verifies that an account has sufficient funds before calling subtractBalance, so a negative balance will not occur.
- Valid response has been provided by the Authentication and Security Module.

11.4.4 Access Routine Semantics

The following apply to every access routine below.

- An exception leaves the state unchanged.

getBalance(userID)

- output: returns the user's current account balance
- exception: NoAccount $\longrightarrow$ no userID found that matches the input

addBalance(userID, amount)

- transition: **amount** is added to the account balance $\longrightarrow$ balance := balance + amount
- output: returns the user's new account balance after **amount** has been added.
- exception: NoAccount $\longrightarrow$ no userID found that matches the input, InvalidAmount $\longrightarrow$ amount < 0

subtractBalance(userID, amount)

- transition: **amount** is removed from the account balance $\longrightarrow$ balance := balance - amount
- output: returns the user's new account balance after **amount** has been subtracted.
- exception: NoAccount $\longrightarrow$ no userID found that matches the input, InvalidAmount $\longrightarrow$ amount < 0

11.4.5 Local Functions

-
- validAmount(amount: BigDecimal) $\longrightarrow \mathbb{B}$: ensures amount is valid (ammount ≥ 0).

12 MIS of Asset Module

This module is M7 in the Module Guide.

12.1 Module

Asset

12.2 Uses

- Persistence Module (Section 7): stores the asset definitions, the number of units of each asset held by users, and the log of changes made by administrators. It prevents entries in the log from being updated or deleted.
- Authentication and Security Module (Section 21): the Principal, UserID and Time types, and requireRole to check that a user is an administrator.

12.3 Syntax

12.3.1 Exported Constants

None. The module uses the following local constant, which is not exported.

Constant	Value	Meaning
MinStartPrice	0.01 FLX	Lowest starting price an asset can have. It equals the Market Simulation Module's MinPrice.

12.3.2 Exported Types

- Principal, UserID and Time: as defined in Section 21
- AssetID = $\mathbb{N}$
- AssetType = $\{\text{STOCK}, \text{BOND}, \text{ETF}, \text{FUND}\}$
- AssetRecord = tuple of (*id*: AssetID, *symbol*: String, *name*: String, *type*: AssetType, *startPrice*: $\mathbb{R}$, *yield*: $\mathbb{R}$, *supply*: $\mathbb{N}$), the definition of an asset. *symbol* is a short code that identifies the asset to users. *startPrice* is its price in FLX when it is first priced. *yield* is its annual dividend yield, where 0 means it pays no dividends. *supply* is the total number of units that exist in the simulated market.
- Asset = tuple of (*id*: AssetID, *symbol*: String, *name*: String, *type*: AssetType, *startPrice*: $\mathbb{R}$, *yield*: $\mathbb{R}$, *supply*: $\mathbb{N}$, *available*: $\mathbb{N} \cup \{0\}$), an AssetRecord together with the number of units not held by any user
- AssetChange = tuple of (*time*: Time, *admin*: UserID, *asset*: AssetID, *before*: AssetRecord $\cup\{\perp\}$, *after*: AssetRecord), a change made by an administrator. *before* = $\perp$ when the asset was added.

An asset can only have one of the types in AssetType. Adding a new type of asset means adding a value to AssetType and calibrating the type in the Market Simulation Module. Neither changes the code that buys and sells assets (PR-SE-02).

12.3.3 Exported Access Programs

Name	In	Out	Exceptions
getAsset	a: AssetID	Asset	UnknownAsset
getAssets	-	sequence of Asset	-
take	a: AssetID, q: $\mathbb{N}$	-	UnknownAsset, InsufficientSupply
release	a: AssetID, q: $\mathbb{N}$	-	UnknownAsset, NotOutstanding
addAsset	p: Principal, s: String, n: String, y: AssetType, sp: $\mathbb{R}$, d: $\mathbb{R}$, sup: $\mathbb{N}$	Asset	Forbidden, InvalidAsset, DuplicateSymbol
updateAsset	p: Principal, a: AssetID, n: String, sup: $\mathbb{N}$, d: $\mathbb{R}$	Asset	Forbidden, UnknownAsset, InvalidAsset
getChanges	p: Principal, *from*: Time, *to*: Time	sequence of AssetChange	Forbidden, InvalidRange

take and release are used only by the Transaction Order Module and are not available through the API. addAsset, updateAsset and getChanges are available only to administrators. An asset's symbol, type and starting price cannot be changed once it is added.

12.4 Semantics

12.4.1 State Variables

- *assets*: set of AssetRecord, the definition of every asset
- *outstanding*: AssetID $\to \mathbb{N} \cup \{0\}$, the number of units of each asset held by users
- *changes*: sequence of AssetChange, every change made by an administrator, in time order. Entries are only ever appended.

When the system is first deployed, $assets = \emptyset$, *outstanding* has no entries and *changes* is empty. All state variables are stored through the Persistence Module, so they keep their values when the backend restarts. The number of units available is not stored. It is calculated from *supply* and *outstanding* whenever it is needed.

12.4.2 Environment Variables

- *clock*: the system clock; *now*() returns the current Time.

12.4.3 Assumptions

- The Authentication and Security Module has authenticated p before every call that takes one.
- Only the Transaction Order Module calls take and release, and only within the unit of work of a trade. Units of work that change the available quantity of the same asset never run at the same time.
- The Market Simulation Module has calibration for every value of AssetType.
- A new asset cannot be traded until it has a price. After addAsset, an administrator or the demonstration data script (TRN-03) calls the Market Simulation Module's backfill for the asset. This module does not call it, because the Market Simulation Module uses this module.
- Assets are never removed.

12.4.4 Access Routine Semantics

The following apply to every access routine below.

- An exception leaves the state unchanged.
- In addAsset, updateAsset and getChanges, requireRole(p, ADMIN) is checked before anything else, and if it fails, Forbidden is raised.

getAsset(a):

- output: $out := view(r)$, where r is the $r \in assets$ with $r.id = a$
- exception: no such r exists $\Rightarrow$ UnknownAsset

getAssets():

- output: $out := view(r)$ for every $r \in assets$, in order of *symbol*

take(a, q):

- transition: $outstanding(a) := outstanding(a) + q$
- exception: a is not an asset $\Rightarrow$ UnknownAsset | $available(a) < q \Rightarrow$ InsufficientSupply

release(a, q):

- transition: $outstanding(a) := outstanding(a) - q$
- exception: a is not an asset $\Rightarrow$ UnknownAsset | $outstanding(a) < q \Rightarrow$ NotOutstanding

Because take and release are called in the same unit of work as the change to the user's holdings, $outstanding(a)$ always equals the total number of units of a held across all portfolios. NotOutstanding is raised only if this has been broken, and causes the trade to be rolled back.

addAsset(p, s, n, y, sp, d, sup):

- transition: let $r = (i, s, n, y, sp, d, sup)$, where i is an AssetID not used by any earlier asset. $assets := assets \cup \{r\}$, $outstanding(i) := 0$, and $(now(), p.user, i, \perp, r)$ is appended to *changes*. These take effect together or not at all.
- output: $out := view(r)$
- exception: $\neg validSymbol(s) \vee n = \text{“”} \vee sp < \mathsf{MinStartPrice} \vee sp$ has more than two decimal places $\vee\ d < 0 \Rightarrow$ InvalidAsset | some $r' \in assets$ has $r'.symbol = s \Rightarrow$ DuplicateSymbol

updateAsset(p, a, n, sup, d):

- Let r be the $r \in assets$ with $r.id = a$, and r' be r with $name = n$, $supply = sup$ and $yield = d$.
- transition: if $r' \neq r$, $assets := (assets - \{r\}) \cup \{r'\}$ and $(now(), p.user, a, r, r')$ is appended to *changes*, together or not at all. If $r' = r$, the state is unchanged and nothing is logged.
- output: $out := view(r')$
- exception: no such r exists $\Rightarrow$ UnknownAsset | $n = \text{“”} \vee d < 0 \vee sup < outstanding(a) \Rightarrow$ InvalidAsset

A new yield takes effect at the Market Simulation Module's next price update. It changes both the asset's next dividend and the drift of its price, so that its expected total return stays the same.

getChanges$(p, from, to)$:

- output: $out :=$ the entries c of *changes* with $from \leq c.time \leq to$, in time order
- exception: $from > to \Rightarrow$ InvalidRange

12.4.5 Local Functions

- *available*: AssetID $\to \mathbb{N} \cup \{0\}$
  $available(a) \equiv r.supply - outstanding(a)$, where r is the $r \in assets$ with $r.id = a$. It is never negative, because take never lets $outstanding(a)$ exceed the supply, and updateAsset never lowers the supply below $outstanding(a)$.

- *view*: AssetRecord → Asset
  $view(r) \equiv (r.id, r.symbol, r.name, r.type, r.startPrice, r.yield, r.supply, available(r.id))$

- *validSymbol*: String → $\mathbb{B}$
  $validSymbol(s) \equiv s$ consists of one to five capital letters A to Z.

13 MIS of Transaction Order Module

This module is M8 in the Module Guide.

13.1 Module

TransactionOrder

13.2 Uses

- Asset Module (Section 12): checks that an asset exists, reads the quantity of it still available, takes units from that quantity for each purchase and returns units to it for each sale. Like the ledger entries, these changes join the unit of work of the trade.

- Market Simulation Module (Section 19): the current price of an asset, through getPrice.

- Currency Ledger Module (Section 11): the user's balance, a debit for each purchase and a credit for each sale. Each debit and credit cites the transaction that caused it as its source (SEC-AUD-01), and joins the unit of work of the trade rather than taking effect on its own.

- Portfolio Module (Section 14): the number of units of an asset a user holds, adding units and their cost for each purchase, and removing units for each sale. Like the ledger entries, these changes join the unit of work of the trade.

- Persistence Module (Section 7): stores quotes and transactions. It provides a unit of work in which a group of writes, made by this module and by the modules above, takes effect together or not at all. It runs the units of work for one user one at a time, and the units of work that change the available quantity of one asset one at a time (PR-RFT-02). It prevents stored transactions from being updated or deleted (SEC-AUD-01).

- Authentication and Security Module (Section 21): the Principal, UserID and Time types.

13.3 Syntax

13.3.1 Exported Constants

None. The module uses the following local constant, which is not exported.

Constant	Value	Meaning
QuoteLifetime	60 seconds	Time after a quote is issued during which it can be executed

13.3.2 Exported Types

- AssetID: as defined by the Asset Module (Section 12)
- Amount and Price: as defined by the Market Simulation Module (Section 19)
- Principal, UserID and Time: as defined in Section 21
- Side = {BUY, SELL}
- Quantity = $\mathbb{N}$, a number of whole units of an asset
- Quote = tuple of (*id*: $\mathbb{N}$, *user*: UserID, *asset*: AssetID, *side*: Side, *qty*: Quantity, *price*: Price, *total*: Amount, *priced*: Time, *expires*: Time). A Quote offers to trade *qty* units at *price* each, for $total = qty \cdot price$. *priced* is the time of the market price it uses.
- Transaction = tuple of (*id*: $\mathbb{N}$, *user*: UserID, *asset*: AssetID, *side*: Side, *qty*: Quantity, *price*: Price, *total*: Amount, *quote*: $\mathbb{N}$, *priced*: Time, *time*: Time). A Transaction records an executed trade, the quote it executed, and the market price it used. *time* is when it was executed.

Because *qty* is a whole number and *price* has two decimal places, *total* is an exact Amount and is never rounded.

13.3.3 Exported Access Programs

Name	In	Out	Exceptions
quote	p: Principal, a: AssetID, s: Side, q: Quantity	Quote	UnknownAsset, NoPrice, InsufficientFunds, InsufficientSupply, InsufficientUnits
execute	p: Principal, id: $\mathbb{N}$	Transaction	UnknownQuote, QuoteExpired, InsufficientFunds, InsufficientSupply, InsufficientUnits
getTransaction	p: Principal, id: $\mathbb{N}$	Transaction	UnknownTransaction
getTransactions	p: Principal, *from*: Time, *to*: Time	sequence of Transaction	InvalidRange

A trade takes two steps. quote prices the order on the server and shows the user what it will cost or pay. execute carries out the quote the user confirms. The app sends only the quote's *id*, never a price, so the trade happens at the price the user confirmed (PR-RFT-02) and that price is always one the server set (SEC-INT-02).

13.4 Semantics

13.4.1 State Variables

- *quotes*: set of Quote, the quotes that have been issued and not yet executed or removed
- *transactions*: sequence of Transaction, every executed trade in time order. Entries are only ever appended.

When the system is first deployed, $quotes = \emptyset$ and *transactions* is empty. Both are stored through the Persistence Module. *transactions* is the audit record of simulated trades required by SEC-AUD-01. Each Transaction refers to the market price it used through (*asset*, *priced*), which identifies an entry in the Market Simulation Module's price history.

13.4.2 Environment Variables

- *clock*: the system clock; *now*() returns the current Time. Tests may supply their own *clock* to check when quotes expire.

13.4.3 Assumptions

- The Authentication and Security Module has authenticated p before every call, and has applied the request rate limit (SEC-IMM-02).
- The backend rejects, with HTTP 400, any request containing a field it does not recognise. A request to execute a trade that also gives a price is therefore rejected (SEC-INT-02).
- Units of work for the same user never run at the same time, so two trades by one user cannot both pass the checks on the same balance or holdings. Units of work that change the available quantity of the same asset never run at the same time, so two purchases cannot both take the last units of an asset.
- The time of the Market Simulation Module's current price for an asset changes only when a new price for that asset is published.
- Every asset defined by the Asset Module can be traded.
- Each user has one portfolio.
- When an account is deleted, the user identifiers in *transactions* are replaced with a pseudonym by the User and Onboarding Module through the Persistence Module, not by this module.

13.4.4 Access Routine Semantics

The following apply to every access routine below.

- $u = p.user$, the user making the request. A user can only quote, execute and view their own trades (SEC-ACC-01).
- Trades have no fees. A purchase costs exactly its *total* and a sale pays exactly its *total*.
- An exception leaves the state unchanged.
- A quote $k \in quotes$ with $now() > k.expires$ may be removed from *quotes* at any time. This has no visible effect, because such a quote can no longer be executed.

quote(p, a, s, q):

- Let pp = getPrice(a) from the Market Simulation Module.
- transition: $quotes := quotes \cup \{k\}$, where $k = (i, u, a, s, q, pp.price, q \cdot pp.price, pp.time, now() +$ QuoteLifetime) and i is an *id* not used by any earlier quote
- output: $out := k$

- exception: a is not an asset $\Rightarrow$ UnknownAsset | a has no price $\Rightarrow$ NoPrice | $s = \text{BUY} \wedge \mathit{balance}(u) < q \cdot \mathit{pp.price} \Rightarrow$ InsufficientFunds | $s = \text{BUY} \wedge \mathit{available}(a) < q \Rightarrow$ InsufficientSupply | $s = \text{SELL} \wedge \mathit{held}(u, a) < q \Rightarrow$ InsufficientUnits

quote checks the balance, the available quantity and the holdings so the user learns of a problem before confirming (BUC 4, BUC 5). execute checks them again, since they may change between the two calls.

execute(p, id):

- If some $t \in \mathit{transactions}$ has $t.\mathit{quote} = \mathit{id} \wedge t.\mathit{user} = u$, the quote has already been executed: $\mathit{out} := t$ and the state is unchanged. This check is made before any other, so repeating a request executes the trade only once.
- Otherwise, let k be the quote in quotes with $k.\mathit{id} = \mathit{id} \wedge k.\mathit{user} = u$.
- transition: let $t = (j, u, k.\mathit{asset}, k.\mathit{side}, k.\mathit{qty}, k.\mathit{price}, k.\mathit{total}, k.\mathit{id}, k.\mathit{priced}, \mathit{now}())$, where j is an id not used by any earlier transaction. Steps 1 to 3 take effect together or not at all.

  1. t is appended to $\mathit{transactions}$.
  2. If $k.\mathit{side} = \text{BUY}$, $k.\mathit{total}$ is debited from u through the Currency Ledger Module, citing $t.\mathit{id}$, and $k.\mathit{qty}$ units of $k.\mathit{asset}$ costing $k.\mathit{total}$ are added to u's holdings through the Portfolio Module, and $k.\mathit{qty}$ units are taken from the available quantity of $k.\mathit{asset}$ through the Asset Module. If $k.\mathit{side} = \text{SELL}$, $k.\mathit{total}$ is credited to u through the Currency Ledger Module, citing $t.\mathit{id}$, $k.\mathit{qty}$ units of $k.\mathit{asset}$ are removed from u's holdings through the Portfolio Module, and $k.\mathit{qty}$ units are returned to the available quantity of $k.\mathit{asset}$ through the Asset Module.
  3. $\mathit{quotes} := \mathit{quotes} - \{k\}$.

  If any step fails, the state is unchanged and the failure is passed to the caller (PR-RFT-02).
- output: $\mathit{out} := t$
- exception: no such k exists $\Rightarrow$ UnknownQuote | $\mathit{expired}(k) \Rightarrow$ QuoteExpired | $k.\mathit{side} = \text{BUY} \wedge \mathit{balance}(u) < k.\mathit{total} \Rightarrow$ InsufficientFunds | $k.\mathit{side} = \text{BUY} \wedge \mathit{available}(k.\mathit{asset}) < k.\mathit{qty} \Rightarrow$ InsufficientSupply | $k.\mathit{side} = \text{SELL} \wedge \mathit{held}(u, k.\mathit{asset}) < k.\mathit{qty} \Rightarrow$ InsufficientUnits

A quote belonging to another user raises UnknownQuote, so a request cannot learn whether another user's quote exists. If the same request is made twice at the same time, the second runs after the first has finished and returns the first's Transaction. The gain or loss realized by a sale (FUNC-10) is calculated by the Portfolio Module, which holds the cost of the units.

getTransaction(p, id):

- output: out := the $t \in transactions$ with $t.id = id \wedge t.user = u$
- exception: no such t exists $\Rightarrow$ UnknownTransaction

getTransactions($p, from, to$):

- output: out := the entries t of $transactions$ with $t.user = u \wedge from \leq t.time \leq to$, in time order (FUNC-11)
- exception: $from > to \Rightarrow$ InvalidRange

13.4.5 Local Functions

- $balance$: UserID $\rightarrow$ Amount
  The user's virtual currency balance, read from the Currency Ledger Module.
- $held$: UserID $\times$ AssetID $\rightarrow \mathbb{N} \cup \{0\}$
  The number of units of the asset the user holds, read from the Portfolio Module.
- $available$: AssetID $\rightarrow \mathbb{N} \cup \{0\}$
  The number of units of the asset still available in the simulated market, read from the Asset Module.
- $expired$: Quote $\rightarrow \mathbb{B}$
  $expired(k) \equiv now() > k.expires \vee priceTime(k.asset) > k.priced$, where $priceTime(a)$ is the $time$ of getPrice(a) from the Market Simulation Module. A quote expires when QuoteLifetime has passed or when a new price for its asset is published, so it can never be executed at a price that is no longer current.

14 MIS of Portfolio Module

This module is M9 in the Module Guide.

14.1 Module

Portfolio

14.2 Uses

[Modules this module uses —SS]

14.3 Syntax

14.3.1 Exported Constants

[Exported constants, or None —SS]

14.3.2 Exported Access Programs

Name	In	Out	Exceptions
[accessProg —SS]	-	-	-

14.4 Semantics

14.4.1 State Variables

[State variables, or None —SS]

14.4.2 Environment Variables

[Environment variables, or None —SS]

14.4.3 Assumptions

[Assumptions, or None —SS]

14.4.4 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

14.4.5 Local Functions

[Local functions, or None —SS]

15 MIS of Leaderboard and Friends Module

This module is M10 in the Module Guide.

15.1 Module

Leaderboard

15.2 Uses

[Modules this module uses —SS]

15.3 Syntax

15.3.1 Exported Constants

[Exported constants, or None —SS]

15.3.2 Exported Access Programs

Name	In	Out	Exceptions
[accessProg —SS]	-	-	-

15.4 Semantics

15.4.1 State Variables

[State variables, or None —SS]

15.4.2 Environment Variables

[Environment variables, or None —SS]

15.4.3 Assumptions

[Assumptions, or None —SS]

15.4.4 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

15.4.5 Local Functions

[Local functions, or None —SS]

16 MIS of Progress Reporting Module

This module is M11 in the Module Guide.

16.1 Module

ProgressReport

16.2 Uses

[Modules this module uses —SS]

16.3 Syntax

16.3.1 Exported Constants

[Exported constants, or None —SS]

16.3.2 Exported Access Programs

Name	In	Out	Exceptions
[accessProg —SS]	-	-	-

16.4 Semantics

16.4.1 State Variables

[State variables, or None —SS]

16.4.2 Environment Variables

[Environment variables, or None —SS]

16.4.3 Assumptions

[Assumptions, or None —SS]

16.4.4 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

16.4.5 Local Functions

[Local functions, or None —SS]

17 MIS of UI Screens Module

This module is M12 in the Module Guide.

17.1 Module

UIScreens

17.2 Uses

[Modules this module uses —SS]

17.3 Syntax

17.3.1 Exported Constants

[Exported constants, or None —SS]

17.3.2 Exported Access Programs

Name	In	Out	Exceptions
[accessProg —SS]	-	-	-

17.4 Semantics

17.4.1 State Variables

[State variables, or None —SS]

17.4.2 Environment Variables

[Environment variables, or None —SS]

17.4.3 Assumptions

[Assumptions, or None —SS]

17.4.4 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

17.4.5 Local Functions

[Local functions, or None —SS]

18 MIS of Notification Module

This module is M13 in the Module Guide.

18.1 Module

Notification

18.2 Uses

[Modules this module uses —SS]

18.3 Syntax

18.3.1 Exported Constants

[Exported constants, or None —SS]

18.3.2 Exported Access Programs

Name	In	Out	Exceptions
[accessProg —SS]	-	-	-

18.4 Semantics

18.4.1 State Variables

[State variables, or None —SS]

18.4.2 Environment Variables

[Environment variables, or None —SS]

18.4.3 Assumptions

[Assumptions, or None —SS]

18.4.4 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

18.4.5 Local Functions

[Local functions, or None —SS]

19 MIS of Market Simulation Module

This module is M14 in the Module Guide.

19.1 Module

MarketSimulation

19.2 Uses

- Asset Module (Section 12): the list of assets, and each asset's type, starting price and annual dividend yield. The yield is read at every tick, because an administrator may change it.
- Currency Ledger Module (Section 11): credits dividend payouts to users' balances.
- Persistence Module (Section 7): stores prices, price history, market events and dividend payouts, and reads the holdings of an asset when a dividend is announced.
- Authentication and Security Module (Section 21): the Time and UserID types.

Holdings are read through the Persistence Module rather than the Portfolio Module, because the Portfolio Module uses this module to value holdings and using it here would create a cycle in the uses hierarchy.

19.3 Syntax

19.3.1 Exported Constants

None. The module uses the following local constants, which are not exported.

Constant	Value	Meaning
TickInterval	15 minutes	Time between scheduled price updates
MinPrice	0.01 FLX	Lowest price an asset can have
DividendInterval	YearLength / 4	Time between an asset's dividend payments (91.3125 days)
YearLength	365.25 days	Length of a year, used to express elapsed time in years

Durations are converted to seconds, the unit of Time, wherever they are combined with a Time. The *step* inputs of getHistory and getTypeTrend are also durations in seconds.

19.3.2 Exported Types

- AssetID and AssetType: as defined by the Asset Module (Section 12), with AssetType = {STOCK, BOND, ETF, FUND}
- UserID and Time: as defined in Section 21
- Amount: a decimal number with exactly two decimal places, in FLX
- Price: an Amount that is at least MinPrice
- PricePoint = tuple of (*asset*: AssetID, *time*: Time, *price*: Price)
- TrendPoint = tuple of (*time*: Time, *value*: $\mathbb{R}$)
- EventKind = {SHOCK, DIVIDEND}
- MarketEvent = tuple of (*id*: $\mathbb{N}$, *time*: Time, *kind*: EventKind, *scope*: AssetID $\cup$ {MARKET}, *magnitude*: $\mathbb{R}$, *description*: String). For a SHOCK, *scope* is MARKET and *magnitude* is the fractional price change of the whole market (negative for a downturn). For a DIVIDEND, *scope* is the asset paying it and *magnitude* is the Amount paid per unit held.

19.3.3 Exported Access Programs

Name	In	Out	Exceptions
init	-	-	-
getPrice	*a*: AssetID	PricePoint	UnknownAsset, NoPrice
getPrices	-	sequence of PricePoint	-
getPriceAt	*a*: AssetID, *t*: Time	PricePoint	UnknownAsset, NoPrice
getHistory	*a*: AssetID, *from*: Time, *to*: Time, *step*: $\mathbb{R}$	sequence of PricePoint	UnknownAsset, InvalidRange
getTypeTrend	*y*: AssetType, *from*: Time, *to*: Time, *step*: $\mathbb{R}$	sequence of TrendPoint	InvalidRange
getEvents	*from*: Time, *to*: Time	sequence of MarketEvent	InvalidRange
tick	-	-	-
backfill	*a*: AssetID, *from*: Time	-	UnknownAsset, AlreadyPriced, InvalidRange

19.4 Semantics

19.4.1 State Variables

- *level*: AssetID $\to \mathbb{R}$, the unrounded price of each asset used by the price model
- *price*: AssetID $\to$ Price, the published price of each asset
- *history*: sequence of PricePoint, every published price in time order. Entries are only ever appended.
- *typeIndex*: AssetType $\to \mathbb{R}$, an index of each asset type's overall price movement
- *trends*: sequence of tuples of (*type*: AssetType, *point*: TrendPoint), every value of *typeIndex* in time order. Entries are only ever appended.
- *events*: sequence of MarketEvent, in time order. Entries are only ever appended.
- *payouts*: set of tuples of (*event*: $\mathbb{N}$, *user*: UserID, *amount*: Amount, *paid*: $\mathbb{B}$), the dividend owed to each holder for each DIVIDEND event
- *lastDividend*: AssetID $\to$ Time, when each asset last paid a dividend
- *lastTick*: Time $\cup\{\perp\}$, when prices were last updated

When the system is first deployed, *level*, *price* and *lastDividend* have no entries, *history*, *trends* and *events* are empty, *payouts* $= \emptyset$, $typeIndex(y) = 100$ for every AssetType y, and $lastTick = \perp$.
All state variables are stored through the Persistence Module, so they keep their values when the backend restarts. *history* and *events* together form the audit record of market price updates and dividends required by SEC-AUD-01.

19.4.2 Environment Variables

- *clock*: the system clock; *now*() returns the current Time.
- *rng*: a source of random numbers, seeded from the backend configuration.
- *scheduler*: calls tick once every TickInterval.
- *calibration*: the price model's parameters, read from a configuration file. For each AssetType y it gives the annual total return μ_y, the annual volatility σ_y, the correlation ρ_y^m with the whole market, the correlation ρ_y^t with other assets of type y, and the sensitivity β_y to market-wide downturns. It also gives *ShockRate*, the expected number of downturns per year, and *ShockSize*, the distribution of their size. The parameters are derived from historical market data that is free to use in an educational application (PR-PA-06).

- *log*: the backend log, to which failed updates are written.

Tests may supply their own *clock* and a fixed seed for *rng*, so that many simulated years can be run in seconds and repeated exactly (test-PR-PA-06). Setting $\sigma_y = 0$, $\mu_y = 0$ and $\mathit{ShockRate} = 0$ keeps prices fixed for tests that need known prices.

19.4.3 Assumptions

- init is called once when the backend starts, before any other access program and before the scheduler first calls tick.
- Every asset defined by the Asset Module has a type, a starting price of at least MinPrice, and an annual dividend yield of at least 0.
- The number of units of an asset that a user holds is a natural number.
- For every AssetType y, $(\rho_y^m)^2 + (\rho_y^t)^2 \leq 1$.
- A single backend instance runs the scheduler, so tick is never run twice at the same time.
- ETFs and funds are priced independently, with their own calibration parameters, rather than from the assets they hold.

19.4.4 Access Routine Semantics

Simulated time runs at the same rate as real time: one simulated year of price movement takes one real year. Every access program that reads state sees the state left by the most recent completed tick.

init():

- transition: ($\mathit{lastTick} = \bot \Rightarrow \mathit{lastTick} := \mathit{now}() \mid \mathit{lastTick} \neq \bot \Rightarrow$ no change)

On first deployment, init starts simulated time. After a restart, the stored state is used unchanged, and the next tick covers the whole time since *lastTick*.

getPrice(a):

- output: *out* := the last entry in *history* whose *asset* is a
- exception: a is not an asset $\Rightarrow$ UnknownAsset | a has no entry in *history* $\Rightarrow$ NoPrice

getPrices():

- output: *out* := getPrice(a) for every asset a that has an entry in *history*

getPriceAt(a, t):

- output: *out* := the last entry in *history* whose *asset* is a and whose *time* is at most t, that is, the price in effect at time t
- exception: a is not an asset $\Rightarrow$ UnknownAsset | no such entry exists $\Rightarrow$ NoPrice

getHistory($a, \mathit{from}, \mathit{to}, \mathit{step}$):

- output: *out* := getPriceAt($a, \mathit{from}+k\cdot\mathit{step}$) for $k = 0, 1, 2, \ldots$ while $\mathit{from}+k\cdot\mathit{step} \le \mathit{to}$, leaving out any time before a's first price
- exception: a is not an asset $\Rightarrow$ UnknownAsset | $\mathit{from} > \mathit{to} \vee \mathit{step} \le 0 \Rightarrow$ InvalidRange

getTypeTrend($y, \mathit{from}, \mathit{to}, \mathit{step}$):

- output: *out* := for $k = 0, 1, 2, \ldots$ while $\mathit{from} + k \cdot \mathit{step} \le \mathit{to}$, the last TrendPoint in *trends* for type y whose *time* is at most $\mathit{from} + k \cdot \mathit{step}$, leaving out any time before the first one
- exception: $\mathit{from} > \mathit{to} \vee \mathit{step} \le 0 \Rightarrow$ InvalidRange

getEvents($\mathit{from}, \mathit{to}$):

- output: *out* := the entries e of *events* with $\mathit{from} \le e.\mathit{time} \le \mathit{to}$, in time order
- exception: $\mathit{from} > \mathit{to} \Rightarrow$ InvalidRange

tick():

- Let $n = \mathit{now}()$ and $\Delta t = (n - \mathit{lastTick})/\mathsf{YearLength}$, the time since the last update in years.
- transition, part 1 (prices, events and dividend announcements). Steps 1 to 5 take effect together or not at all.
  1. Downturn: with probability $1 - e^{-\mathit{ShockRate}\cdot\Delta t}$, a SHOCK event with scope MARKET and magnitude J drawn from *ShockSize* is appended to *events*. Otherwise $J = 0$. This is the probability that at least one downturn occurs within Δt when downturns arrive at random at an average rate of *ShockRate* per year, as in the jump model of **?**.
  2. Prices: one market draw Z_m, one draw Z_y for each AssetType y and one draw Z_a for each asset a are taken with *drawNormal*. For each asset a of type y that already has a price, $\mathit{level}(a) := \mathit{level}(a)\cdot e^{\mathit{logReturn}(a,\Delta t,Z_m,Z_y,Z_a)}\cdot(1+\beta_y J)$. An asset with no price yet starts with $\mathit{level}(a) := \mathit{startPrice}(a)$ and $\mathit{lastDividend}(a) := n$. Then $\mathit{price}(a) := \max(\mathsf{MinPrice}, \mathit{round}_2(\mathit{level}(a)))$ and $(a, n, \mathit{price}(a))$ is appended to *history*.

3. Type trends: for each AssetType y with at least one priced asset, $typeIndex(y)$ is multiplied by the average, over the assets of type y priced in step 2, of $level(a)$ after the step divided by $level(a)$ before it. $(y, (n, typeIndex(y)))$ is appended to *trends*.
4. Dividend announcements: for each asset a with $yield(a) > 0$ whose last dividend, at $lastDividend(a)$, was at least DividendInterval before n, let f be DividendInterval in years (here $\frac{1}{4}$) and $d = round_2(price(a) \cdot yield(a) \cdot f)$, the amount paid per unit. If $d \geq 0.01$, a DIVIDEND event e with scope a and magnitude d is appended to *events*, and for every $(u, q) \in holdings(a)$ with $q > 0$, $(e.id, u, q \cdot d, \text{false})$ is added to *payouts*. In either case $lastDividend(a) := n$.
5. $lastTick := n$.

If any of steps 1 to 5 fails, the state is unchanged and the failure is written to *log*. The last valid prices stay in effect, and the next tick covers the whole time since *lastTick* (BUC 6).

- transition, part 2 (dividend payouts): for each $p \in payouts$ with $\neg p.paid$, $p.amount$ is credited to $p.user$ through the Currency Ledger Module, citing event $p.event$ as its source, and $p.paid := \text{true}$. Each payout's credit and its change to *paid* take effect together or not at all. A payout that fails stays unpaid and is retried at the next tick, so each payout is credited exactly once.

backfill($a, from$):

- transition: $level(a) := startPrice(a)$. Then, for each time $t = from + k \cdot$ TickInterval with $k = 1, 2, \ldots$ and $t \leq now()$, $level(a)$ is updated as in step 2 of tick with $\Delta t =$ TickInterval/YearLength and J drawn as in step 1 of tick, and $(a, t, price(a))$ is appended to *history*. Finally $lastDividend(a) := now()$. No events or payouts are created.

- exception: a is not an asset $\Rightarrow$ UnknownAsset | a has an entry in *history* $\Rightarrow$ AlreadyPriced | $from \geq now() \Rightarrow$ InvalidRange

backfill gives a newly listed asset a price history, and is used by the demonstration data script (TRN-03).

19.4.5 Local Functions

- *type*: AssetID $\to$ AssetType, *startPrice*: AssetID $\to$ Price, and *yield*: AssetID $\to \mathbb{R}$
  The asset's type, starting price and annual dividend yield, read from the Asset Module.

- *holdings*: AssetID $\to$ set of (UserID $\times \mathbb{N}$)
  $holdings(a) \equiv$ each user holding asset a and the number of units they hold, read through the Persistence Module.

- $round_2$: $\mathbb{R} \to$ Amount
  $round_2(x) \equiv x$ rounded to two decimal places, with ties rounded to the even digit.

- *drawNormal*: $\to \mathbb{R}$
  A random number from the standard normal distribution, taken from *rng*.

- *logReturn*: AssetID $\times\mathbb{R} \times \mathbb{R} \times \mathbb{R} \times \mathbb{R} \to \mathbb{R}$
  For an asset a of type y with $q = \mathit{yield}(a)$:

$$\mathit{logReturn}(a, \Delta t, Z_m, Z_y, Z_a) \equiv \left(\mu_y - q - \lambda\kappa_y - \frac{\sigma_y^2}{2}\right)\Delta t + \sigma_y\sqrt{\Delta t}\, W$$

  where $\lambda = \mathit{ShockRate}$, $\kappa_y = \beta_y \cdot E[J]$ with $E[J]$ the mean of *ShockSize*, and W combines the market, type and asset draws:

$$W = \rho_y^m Z_m + \rho_y^t Z_y + \sqrt{1 - (\rho_y^m)^2 - (\rho_y^t)^2}\, Z_a$$

  This is geometric Brownian motion, the standard model of stock price movement (**??**). The yield is subtracted from the drift so that an asset's price growth plus its dividends matches the calibrated total return μ_y, following the treatment of continuous dividend yields in **?**. W splits each asset's random movement into a part shared by the whole market, a part shared by its asset type and a part of its own, extending the single-index model of **?**. The square root keeps the variance of W at 1, so σ_y remains the asset's volatility. Market-wide downturns are added as jumps in tick. They occur λ times a year on average and change a type-y asset's price by κ_y each time on average ($\kappa_y < 0$), so the term $\lambda\kappa_y$ raises the drift by the same amount and keeps the expected total return at μ_y, as in the jump model of **?**.

20 MIS of API Client Module

This module is M15 in the Module Guide.

20.1 Module

ApiClient

20.2 Uses

This module runs on the phone, and mobile modules reach the backend only through it. It uses every backend module whose access programs a user of the app can request, together with those programs' types and exceptions:

- User and Onboarding Module (Section 8)
- Activity Module (Section 9)
- Goals Module (Section 10)
- Currency Ledger Module (Section 11)
- Asset Module (Section 12)
- Transaction Order Module (Section 13)
- Portfolio Module (Section 14)
- Leaderboard and Friends Module (Section 15)
- Progress Reporting Module (Section 16)
- Notification Module (Section 18)
- Market Simulation Module (Section 19)
- Authentication and Security Module (Section 21), which also provides the session renewal this module performs on its own

20.3 Syntax

20.3.1 Exported Constants

None. The module uses the following local constants, which are not exported.

Constant	Value	Meaning
RequestTimeout	10 seconds	Time after which a request with no response counts as failed
MaxRetries	2	Number of times a failed request is sent again
RetryBaseDelay	0.5 seconds	Wait before the first retry; the wait doubles for each further retry

20.3.2 Exported Types

The inputs and outputs of the exported programs use the types defined in the sections of the backend modules listed under Uses. In addition:

- Time, DeviceID, Token and MfaChallenge: as defined in Section 21
- SignInOutcome = {SIGNED_IN}∪ MfaChallenge
- For any type T, Fresh(T) = tuple of (*data*: T, *receivedAt*: Time, *stale*: $\mathbb{B}$). A result of this type is returned by every program that only reads data. *stale* is true when *data* is a stored copy rather than a response just received from the backend, so it may be out of date.
- ReadOp: the set of exported programs that only read data

20.3.3 Exported Access Programs

For each access program of a backend module listed under Uses, this module exports an access program with the same name, unless it is one of the backend-only programs listed below or is specified individually in the table. Its inputs are the backend program's inputs without the Principal, the client IP address and the DeviceID, which this module supplies itself. Its output is the backend program's output, wrapped in Fresh if the program only reads data. Its exceptions are the backend program's exceptions plus the transport exceptions listed below.

The following backend programs are not exported by this module. All but refresh are used only within the backend; refresh is requested by this module itself to renew the session (rule 2 below). When the section of another backend module defines a program of this kind, it is added here.

Backend Module	Programs Not Exported
Market Simulation	init, tick, backfill
Authentication and Security	init, hashPassword, authenticate, authorize, requireRole, confirmPassword, refresh, endAllSessions

The following programs are specified individually, because they keep the session and stored data on the phone or differ from the backend program of the same name.

Name	In	Out	Exceptions
init	-	-	-
signIn	*email*: String, *pw*: String	SignInOutcome	UnknownAccount, AccountLocked, InvalidCredentials
verifyMfa	*cid*: Token, *code*: String	-	InvalidCode
signOut	-	-	-
isSignedIn	-	$\mathbb{B}$	-
peek	*op*: ReadOp, *args*: the inputs of *op*	Fresh(T) $\cup\{\bot\}$, where T is the data type of *op*'s output	-

In addition to their other exceptions, signIn, verifyMfa and every program exported by the rule above may raise the following transport exceptions:

- NetworkError: the backend could not be reached, and no stored copy of the requested data is available.
- ServerError: the backend failed because of a fault of its own.
- RateLimited: the backend refused the request because too many requests were made; it carries the time to wait before trying again.
- SessionExpired: the program needs a signed-in user and the session could not be renewed, so the user must sign in again. Raised only by programs whose backend program needs an authenticated user; the assumptions of Section 21 list the requests that do not.

20.4 Semantics

20.4.1 State Variables

- *deviceId*: DeviceID, created once for this installation
- *accessToken*: Token $\cup\{\bot\}$, the current access token, held in memory only
- *refreshToken*: Token $\cup\{\bot\}$, the current refresh token, held in *secureStore*
- *cache*: (ReadOp $\times$ inputs) $\to$ (data, Time), the last data received for each read and when it was received, held in *localStore*
- *renewing*: $\mathbb{B}$, true while a session renewal is in progress

20.4.2 Environment Variables

- *backend*: the backend service, reached over the network at the address given in the app's build configuration.
- *secureStore*: encrypted storage on the device that other apps cannot read.
- *localStore*: storage on the device that is private to the app.
- *clock*: the device clock; *now*() returns the current Time.

20.4.3 Assumptions

- init is called once, when the app starts, before any other access program.
- In release builds the backend is reached only over TLS 1.3 (SEC-PRV-01), and a backend address that does not use it is refused.
- The backend treats requests that carry the same request key as one request: it performs the action once and returns the same result for each.
- Only one user is signed in on an installation at a time.
- Amounts and prices are received as exact decimal values, and this module never converts them to floating-point numbers (PR-PA-01). The backend calculates all amounts and prices (SEC-INT-02).

20.4.4 Access Routine Semantics

Every request to the backend follows these rules.

1. Credentials: a request for a program that needs a signed-in user carries *accessToken*. Every request carries *deviceId*.
2. Renewal: if the backend rejects a request with Unauthenticated, the session is renewed by requesting the backend's refresh with *refreshToken*. Requests rejected while *renewing* is true wait for that renewal instead of starting another, because each renewal replaces the refresh token. The renewal ends in one of three ways:
    - refresh returns a TokenPair $r \Rightarrow$ *accessToken* := *r.access* and *refreshToken* := *r.refresh*, the new refresh token is stored in *secureStore*, and the request is sent again once.
    - refresh raises Unauthenticated, or there is no refresh token $\Rightarrow$ both tokens are set to $\perp$, *cache* := $\emptyset$, and SessionExpired is raised.
    - refresh gets no response or the backend reports a fault of its own $\Rightarrow$ the tokens and *cache* are unchanged, and the request counts as failed: a read falls back as in rule 3, and a write raises NetworkError or ServerError.

3. Reads: if the backend gives no response within RequestTimeout or reports a fault of its own, the request is sent again up to MaxRetries times, waiting $backoff(k)$ before retry k. When data d is received, $cache(op, args) := (d, now())$ if $cacheable(op)$, and the output is $(d, now(), \text{false})$. If every attempt fails: ($cache(op, args)$ exists $\Rightarrow$ its data and time with $stale = \text{true}$ | otherwise $\Rightarrow$ NetworkError or ServerError). This keeps the app usable with the last data received when the backend cannot be reached (PR-RFT-04).

4. Writes: each call creates a new request key with $newRequestKey$, and every retry of that call reuses it. Retries follow rule 3, so a write whose response was lost takes effect only once (PR-RFT-02). A write that fails is never stored to be sent later.

5. RateLimited is raised as soon as it is received, with the wait time given by the backend, and the request is not retried.

6. Any other exception from the backend is raised under the same name.

init():

- transition:
  - $deviceId$:= the DeviceID held in $secureStore$. If there is none, a new one is created with $newDeviceId()$ and stored in $secureStore$.
  - $refreshToken$:= the token held in $secureStore$, or $\perp$ if there is none.
  - $accessToken := \perp$.

signIn($email, pw$):

- transition: if the backend returns a TokenPair r, then $accessToken := r.access$, and $refreshToken := r.refresh$ is stored in $secureStore$.
- output: out := (the backend returns a TokenPair $\Rightarrow$ SIGNED_IN | the backend returns an MfaChallenge $c \Rightarrow c$)
- exception: the backend's exceptions, as in Section 21

verifyMfa($cid, code$):

- transition: $accessToken$ and $refreshToken$ are set from the returned TokenPair, as in signIn
- exception: the backend's exceptions, as in Section 21

signOut():

- transition: the backend's signOut is requested once and its result is ignored. Then $accessToken, refreshToken := \perp, \perp$, the refresh token is removed from $secureStore$, and $cache := \emptyset$. The device is signed out even when the backend cannot be reached.

isSignedIn():

- output: $out := refreshToken \neq \bot$

Every program $X(args)$ exported by the rule:

- transition and output: the backend program X is requested with *args*. If X only reads data, the request is a read and *out* is given by rule 3. Otherwise the request is a write (rule 4), and *out* is the backend program's output.
- exception: the exceptions of the backend program X, as given in its module's section, and the transport exceptions

peek(op, *args*):

- output: $out := (cache(op, args)$ exists $\Rightarrow$ its data and time with $stale = \text{true} \mid \text{otherwise} \Rightarrow \bot)$

peek sends no request. It lets a screen show the last data received as soon as the app opens, while it requests current data with *op* (PR-SL-01).

20.4.5 Local Functions

- *backoff*: $\mathbb{N} \to \mathbb{R}$
  $backoff(k) \equiv \mathsf{RetryBaseDelay} \cdot 2^{k-1}$ seconds
- *cacheable*: ReadOp $\to \mathbb{B}$
  $cacheable(op) \equiv$ the output of *op* contains no profile details and no health or fitness data, which must be encrypted when stored (SEC-PRV-01). Other data, such as balances, holdings and prices, is cached.
- *newRequestKey*: $\to$ String
  A random value that has not been used before.
- *newDeviceId*: $\to$ DeviceID
  A random value that identifies this installation.

21 MIS of Authentication and Security Module

This module is M16 in the Module Guide.

21.1 Module

Auth

21.2 Uses

- Persistence Module (Section 7): stores user records, sessions, sign-in lockout state, multi-factor authentication challenges and password reset tokens.

Auth runs on the backend only. Storing tokens on the phone and attaching them to requests is the responsibility of the API Client Module (Section 20).

21.3 Syntax

21.3.1 Exported Constants

The following constants are defined in the SRS.

Constant	Value	SRS Requirement
AccessExpiry	60 minutes	SEC-ACC-01
SessionTimeout	30 days without use	SEC-ACC-05
MinPasswordLength	8 characters	SEC-ACC-04
LockoutAttempts	5 consecutive failures	SEC-ACC-04
LockoutDuration	15 minutes	SEC-ACC-04
RateLimit	60 requests per minute	SEC-IMM-02

Two further constants are local to this module and are not exported: MfaCodeExpiry = 10 minutes, the time a multi-factor authentication code remains valid, and ResetTokenExpiry = 30 minutes, the time a password reset link remains valid.

21.3.2 Exported Types

- UserID = $\mathbb{N}$
- SessionID = $\mathbb{N}$
- DeviceID = String, an identifier the client generates when the app is installed
- Token = String

- Time = $\mathbb{R}$, seconds since the Unix epoch
- Role = $\{\text{STANDARD}, \text{ADMIN}\}$
- Principal = tuple of (*user*: UserID, *role*: Role, *session*: SessionID, *device*: DeviceID)
- TokenPair = tuple of (*access*: Token, *refresh*: Token, *accessExpires*: Time)
- MfaChallenge = tuple of (*challengeId*: Token)
- SignInResult = TokenPair $\cup$ MfaChallenge

21.3.3 Exported Access Programs

Name	In	Out	Exceptions
init	-	-	-
hashPassword	*pw*: String	String	WeakPassword
signIn	*email*: String, *pw*: String, *dev*: DeviceID, *ip*: String	SignInResult	RateLimited, UnknownAccount, AccountLocked, InvalidCredentials
verifyMfa	*cid*: Token, *code*: String, *ip*: String	TokenPair	RateLimited, InvalidCode
refresh	*rt*: Token	TokenPair	Unauthenticated
authenticate	*tok*: Token, *ip*: String	Principal	Unauthenticated, RateLimited
authorize	*p*: Principal, *owner*: UserID	-	Forbidden
requireRole	*p*: Principal, *r*: Role	-	Forbidden
confirmPassword	*p*: Principal, *pw*: String	-	InvalidCredentials
signOut	*p*: Principal	-	-
endAllSessions	*u*: UserID	-	-
setMfaEnabled	*p*: Principal, *on*: $\mathbb{B}$	-	-
requestPasswordReset	*email*: String, *ip*: String	-	RateLimited
resetPassword	*t*: Token, *pw*: String	-	InvalidResetToken, WeakPassword

RateLimited carries the time the client must wait before its next request will be accepted.

21.4 Semantics

21.4.1 State Variables

- *sessions*: set of Session, where Session = tuple of (*id*: SessionID, *user*: UserID, *device*: DeviceID, *refreshHash*: String, *lastUsed*: Time, *revoked*: $\mathbb{B}$) (initially $\emptyset$)
- *failedAttempts*: UserID $\to \mathbb{Z}$, the number of consecutive failed sign-in attempts (initially 0)
- *lockedUntil*: UserID $\to$ Time, the time until which sign-in is locked (initially 0)
- *mfaEnabled*: UserID $\to \mathbb{B}$ (initially false)
- *challenges*: set of tuples of (*id*: Token, *user*: UserID, *device*: DeviceID, *codeHash*: String, *expires*: Time) (initially $\emptyset$)
- *resetTokens*: set of tuples of (*tokenHash*: String, *user*: UserID, *expires*: Time) (initially $\emptyset$)
- *requestLog*: String $\to$ sequence of Time, the times of recent requests, keyed by client IP address or by user ID
- *commonPasswords*: set of String, the common passwords that are rejected (SEC-ACC-04)

The initial values above apply when the system is first deployed. All state variables except *requestLog* and *commonPasswords* are stored through the Persistence Module, so they keep their values when the backend restarts. *requestLog* and *commonPasswords* are held in memory and are set by init.

21.4.2 Environment Variables

- *clock*: the system clock; *now*() returns the current Time.
- *signingKey*: the secret key used to sign access tokens. It is read from the deployment environment and is never stored in the project repository (SEC-ACC-03).
- *mailServer*: the outgoing email server used to send multi-factor authentication codes and password reset links.
- *securityLog*: sequence of tuples of (*time*: Time, *ip*: String $\cup\{\perp\}$, *event*: String, *user*: UserID $\cup\{\perp\}$), the log to which security events are written (SEC-AUD-02). Entries are only ever appended.
- *passwordFile*: the file listing common passwords, read by init.

21.4.3 Assumptions

- init is called once when the backend starts, before any other access program.
- Client–server communication uses TLS 1.3, configured when the backend is deployed (SEC-PRV-01). This module does not enforce it.
- The User and Onboarding Module ensures that email addresses are unique.
- Each client sends the same DeviceID on every request from a given installation.
- A single backend instance is running, so the in-memory *requestLog* counts every request. Running several instances would require a shared store.
- authenticate is called before any other backend access program on every protected request. Registration, signIn, verifyMfa, refresh, requestPasswordReset, resetPassword and the health check are the only unprotected requests.

21.4.4 Access Routine Semantics

The following apply to every access routine below.

- Request counting: each call to signIn, verifyMfa or requestPasswordReset first appends $now()$ to $requestLog(ip)$. In authenticate, once the token is verified, $now()$ is appended to $requestLog(p.user)$.
- Rate limits: when RateLimited is raised for key k, it carries $waitTime(k)$.
- Security log: every exception raised by this module, and every account lockout, appends the entry $(now(), ip, e, u)$ to *securityLog*. Here ip is the routine's ip input, or $\perp$ if it has none; e is the name of the exception, or LOCKOUT; and u is the user concerned, or $\perp$ if none is known. Passwords, codes and tokens are never written.
- An exception leaves the state unchanged apart from these entries, unless a transition for that exception is given.

init():

- transition:
  - *requestLog* := the mapping of every key to the empty sequence
  - *commonPasswords* := the passwords listed in *passwordFile*

hashPassword(pw):

- output: $out := hash(pw)$
- exception: $isWeak(pw) \Rightarrow$ WeakPassword

signIn($email, pw, dev, ip$):

- Let $a = \mathit{findUser}(email)$, $ok = \mathit{matches}(pw, a.\mathit{passwordHash})$, $rt = \mathit{newSecret}()$, $c = \mathit{newCode}()$, and $s = (\mathit{newId}(), a.id, dev, \mathit{digest}(rt), \mathit{now}(), \text{false})$.
- transition:
  - $ok \wedge \mathit{mfaEnabled}(a.id) \Rightarrow \mathit{failedAttempts}(a.id) := 0$. A new challenge with id $\mathit{newSecret}()$, user $a.id$, device dev, code hash $\mathit{hash}(c)$ and expiry $\mathit{now}() +$ MfaCodeExpiry is added to $\mathit{challenges}$. The code c is emailed to the user's address $a.\mathit{email}$ through the $\mathit{mailServer}$.
  - $ok \wedge \neg \mathit{mfaEnabled}(a.id) \Rightarrow \mathit{failedAttempts}(a.id) := 0$, $\mathit{sessions} := \mathit{sessions} \cup \{s\}$
  - When InvalidCredentials is raised, $\mathit{failedAttempts}(a.id)$ increases by 1. If it then equals LockoutAttempts, the account is locked: $\mathit{lockedUntil}(a.id) := \mathit{now}() +$ LockoutDuration and $\mathit{failedAttempts}(a.id) := 0$.
- output: $\mathit{out} := (\mathit{mfaEnabled}(a.id) \Rightarrow (\mathit{id} \text{ of the new challenge}) \mid \neg \mathit{mfaEnabled}(a.id) \Rightarrow (\mathit{issueAccess}(s), rt, \mathit{now}() + \mathsf{AccessExpiry}))$
- exception, with the cases checked in order:
  - $\mathit{overLimit}(ip) \Rightarrow$ RateLimited
  - $a = \bot \Rightarrow$ UnknownAccount
  - $\mathit{now}() < \mathit{lockedUntil}(a.id) \Rightarrow$ AccountLocked
  - $\neg ok \Rightarrow$ InvalidCredentials

verifyMfa($cid, code, ip$):

- Let h be the challenge in $\mathit{challenges}$ with $h.id = cid$, $rt = \mathit{newSecret}()$, and $s = (\mathit{newId}(), h.\mathit{user}, h.\mathit{device}, \mathit{digest}(rt), \mathit{now}(), \text{false})$.
- transition: $\mathit{challenges} := \mathit{challenges} - \{h\}$, $\mathit{sessions} := \mathit{sessions} \cup \{s\}$
- output: $\mathit{out} := (\mathit{issueAccess}(s), rt, \mathit{now}() + \mathsf{AccessExpiry})$
- exception: $\mathit{overLimit}(ip) \Rightarrow$ RateLimited $\mid$ h does not exist $\vee$ $\mathit{now}() > h.\mathit{expires} \vee \neg \mathit{matches}(code, h.\mathit{codeHash}) \Rightarrow$ InvalidCode

refresh(rt):

- Let s be the session in $\mathit{sessions}$ with $\neg s.\mathit{revoked} \wedge s.\mathit{refreshHash} = \mathit{digest}(rt)$, and $rt' = \mathit{newSecret}()$.
- transition: $s.\mathit{refreshHash}, s.\mathit{lastUsed} := \mathit{digest}(rt'), \mathit{now}()$. The old refresh token can no longer be used.

- output: $out := (\mathit{issueAccess}(s), rt', \mathit{now}() + \mathsf{AccessExpiry})$
- exception: s does not exist $\lor\ \mathit{now}() - s.\mathit{lastUsed} > \mathsf{SessionTimeout} \Rightarrow$ Unauthenticated

authenticate($\mathit{tok}, \mathit{ip}$):

- Let $p = \mathit{verifyAccess}(\mathit{tok})$ and s be the session in $\mathit{sessions}$ with $s.\mathit{id} = p.\mathit{session}$.
- output: $out := p$
- exception, with the cases checked in order:
  - $p = \bot \lor s.\mathit{revoked} \Rightarrow$ Unauthenticated
  - $\mathit{overLimit}(p.\mathit{user}) \Rightarrow$ RateLimited

authorize(p, owner):

- exception: $p.\mathit{user} \neq \mathit{owner} \Rightarrow$ Forbidden

requireRole(p, r):

- exception: $p.\mathit{role} \neq r \Rightarrow$ Forbidden

confirmPassword(p, pw):

- exception: $\neg \mathit{matches}(pw, \mathit{account}(p.\mathit{user}).\mathit{passwordHash}) \Rightarrow$ InvalidCredentials

signOut(p):

- transition: $s.\mathit{revoked} :=$ true for the session s with $s.\mathit{id} = p.\mathit{session}$. Every access and refresh token issued for that session, and so for that device, is rejected from then on.

endAllSessions(u):

- transition: $s.\mathit{revoked} :=$ true for every $s \in \mathit{sessions}$ with $s.\mathit{user} = u$; $\mathit{challenges} := \{h \in \mathit{challenges} \mid h.\mathit{user} \neq u\}$; $\mathit{resetTokens} := \{r \in \mathit{resetTokens} \mid r.\mathit{user} \neq u\}$

setMfaEnabled(p, on):

- transition: $\mathit{mfaEnabled}(p.\mathit{user}) := \mathit{on}$

requestPasswordReset($\mathit{email}, \mathit{ip}$):

- Let $a = \mathit{findUser}(\mathit{email})$ and $t = \mathit{newSecret}()$.
- transition:
  - $a \neq \bot \Rightarrow$ the reset token $(\mathit{digest}(t), a.\mathit{id}, \mathit{now}() + \mathsf{ResetTokenExpiry})$ is added to $\mathit{resetTokens}$, and a link containing t is emailed to $a.\mathit{email}$ through $\mathit{mailServer}$
  - $a = \bot \Rightarrow$ no change

- output: none in either case
- exception: $overLimit(ip) \Rightarrow$ RateLimited

resetPassword(t, pw):

- Let r be the reset token in *resetTokens* with $r.tokenHash = digest(t)$.
- transition:
  - $account(r.user).passwordHash := hash(pw)$
  - $failedAttempts(r.user) := 0$ and $lockedUntil(r.user) := 0$, so a locked-out user can sign in again
  - the transition of endAllSessions($r.user$), which also removes r so the link cannot be used again
- exception: r does not exist $\vee$ $now() > r.expires \Rightarrow$ InvalidResetToken | $isWeak(pw) \Rightarrow$ WeakPassword

21.4.5 Local Functions

- UserRecord = tuple of (*id*: UserID, *email*: String, *role*: Role, *passwordHash*: String)
- *findUser*: String $\to$ UserRecord $\cup\{\bot\}$
  $findUser(e) \equiv$ the stored user record whose email is e, read through the Persistence Module, or $\bot$ if there is none.
- *account*: UserID $\to$ UserRecord
  $account(u) \equiv$ the stored user record whose id is u, read and written through the Persistence Module.
- *isWeak*: String $\to \mathbb{B}$
  $isWeak(pw) \equiv |pw| <$ MinPasswordLength $\vee$ $pw \in commonPasswords$
- *hash*: String $\to$ String
  A deliberately slow, salted one-way hash using a new random salt on each call. It is used for passwords and multi-factor authentication codes.
- *matches*: String $\times$ String $\to \mathbb{B}$
  $matches(x, h) \equiv h$ is the result of $hash(x)$ under the salt stored in h.
- *digest*: String $\to$ String
  A fast, unsalted one-way hash. It is used only for long random values produced by *newSecret*, so that a stored value can be looked up from the token the client presents.

- *newSecret*: $\rightarrow$ Token
  A random value of at least 256 bits from a cryptographically secure random number generator.

- *newCode*: $\rightarrow$ String
  A random six-digit code from a cryptographically secure random number generator.

- *newId*: $\rightarrow \mathbb{N}$
  An identifier not used by any existing session.

- *issueAccess*: Session $\rightarrow$ Token
  $issueAccess(s) \equiv$ a token containing $s.user$, $account(s.user).role$, $s.id$, $s.device$ and the expiry time $now() + \mathsf{AccessExpiry}$, signed with $signingKey$.

- *verifyAccess*: Token $\rightarrow$ Principal $\cup\{\bot\}$
  $verifyAccess(t) \equiv \bot$ if t is missing, its signature does not match $signingKey$, or its expiry time is not after $now()$; otherwise the Principal contained in t.

- *overLimit*: String $\rightarrow \mathbb{B}$
  $overLimit(k) \equiv |\langle x \in requestLog(k) \mid x > now() - 60 \rangle| > \mathsf{RateLimit}$

- *waitTime*: String $\rightarrow \mathbb{R}$
  $waitTime(k) \equiv$ the smallest number of seconds $w \geq 0$ for which $requestLog(k)$ has fewer than RateLimit entries later than $now() + w - 60$. A request for k made w seconds from now is accepted.

22 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

22.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

22.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

22.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

23 Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?
4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?

5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)

6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

7. (After you have implemented another team's module, which means this isn't filled in until after the original deadline). What did you learn by implementing another team's module? Were all the details you needed in the documentation, or did you need to make assumptions, or ask the other team questions? If your team also had another team implement one of your modules, what was this experience like? Are there things in your documentation you could have changed to make the process go more smoothly for when an "outsider" completes some of the implementation?

   Your reflection should be about concepts learned in your SE education, such as modular decomposition, cohesion, coupling, information hiding, etc. You should critique the quality and utility of the specification you received.

   You should create a PR for your implemented module. A link to the PR should be part of your reflection.